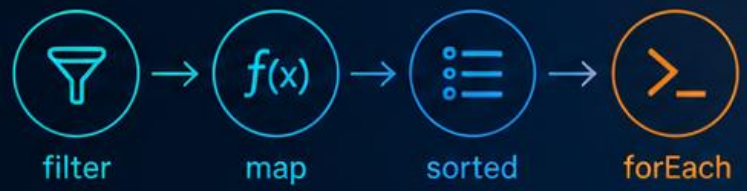
Programación funcional con Java

```java
List<String> nombres = List.of("Ana", "Luis", "María", "Diego");
nombres.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .sorted())
    .forEach(System.out::println);
```



Funciones. Expresiones. Inmutabilidad. Composición. Streams. Rendimiento.





INMUTABILIDAD | COMPOSICIÓN | RENDIMIENTO | CÓDIGO LIMPIO

INTERFACES FUNCIONALES

- Es una interfaz con un y sólo un método abstracto.
- Puede tener todos los métodos default, static o private que quiera.
- Se recomienda anotarla con @FunctionalInterface

```
@FunctionalInterface
public interface MiInterfazFuncional {

    void procesa(Cliente c);

}
```

java.util.function

Tipos de Interfaces funcionales	Interfaces	Método abstracto
Predicados	Predicate<T>	boolean test(T)
Representa un predicado, devuelve un booleano a partir del parámetro		
Consumidores	Consumer<T>	void accept(T)
Representa una operación que acepta un parámetro y no retorna nada		
Funciones	Function<T, R>	R apply(T)
Representa una función, acepta un argumento y produce un resultado		
Proveedores	Supplier<R>	R get()
Representa un proveedor de un resultado		
Operadores	UnaryOperator<T>	T apply(T)
Representa una función, acepta un argumento y produce un resultado del mismo tipo		

Expresiones lambda

- Una característica única de las interfaces funcionales es que pueden ser instanciadas usando “expresiones lambdas”

- Permiten crear código más conciso y significativo, además de abrir la puerta hacia la programación funcional en Java, en donde las funciones juegan un papel fundamental

- Funciones como entidades de primer nivel
  - Las expresiones lambda permiten utilizar las funciones (métodos) como se utilizan los tipos primitivos o los objetos. Es decir, poder pasar funciones, en tiempo de ejecución como valores de variables, valores de retorno o parámetros de otras funciones.

Expresiones lambda - sintaxis

- Sintaxis general

  (lista parámetros) -> {cuerpo de la expresión lambda}

Lista de parámetros

Si hay un solo parámetro, no son obligatorios los paréntesis

Si no hay parámetro o hay más de uno, son obligatorios

Cuerpo de la expresión lambda

Si tiene una sólo línea, no es necesario utilizar las llaves y en el caso que deba devolver algo no es necesario el return

Cuando tiene más de una línea, las llaves son obligatorias y en el caso que deba devolver un valor debe incluir return

Expresiones lambda - ejemplos

- x -> x + 2
- () -> System.out.println(“mensaje”)
- (lado1, lado2) -> lado1 * lado2
- (int lado1, int lado2) -> {return lado1 * lado2;}

Referencia a métodos

- Permiten utilizar un método como expresión lambda, simplificando aún más la sintaxis

  System.out :: println en lugar de **s -> System.out.println(s)**

  El lambda representa un consumer y el método println(…) es un método consumer.

- Tipos
  - Referencia a métodos estáticos
  - Referencia a métodos de instancia de un objeto existente
  - Referencia a métodos de instancia de un tipo
  - Referencia a métodos constructores

API Stream (java.util.stream)

- Este nuevo API provee utilidades para para permitir operaciones "estilo programación funcional" sobre flujo de valores. La forma más simple de obtener un Stream seguramente sea a partir de una colección.

- Permite realizar operaciones de tipo filtro/mapeo/reducción sobre colecciones de datos de forma secuencial o paralela y que su implementación sea transparente para el desarrollador.

API Stream (java.util.stream)

- Stream se define como una secuencia de elementos que provienen de una fuente que soporta operaciones para el procesamiento de sus datos:
  - De forma declarativa usando expresiones lambda.
  - Permitiendo el posible encadenamiento de varias operaciones, con lo que se logra tener un código fácil de leer y con un objetivo claro.
  - De forma secuencial o paralela (Fork/Join).
- Permite realizar operaciones sobre el flujo de datos usando el modelo filtro/mapeo/reducción:
  - los datos que se van a procesar (filtro)
  - se convierten a otro tipo de dato (mapeo)
  - y al final se obtiene el resultado deseado (reducción)

API Stream (java.util.stream)

- Las operaciones intermedias retornan otro Stream (Method chaining)
- Estas se encolan hasta que se invoca una operación terminal
- Un stream sólo puede recorrerse una sola vez (IllegalStateException)

```
Stream<String> stream = List.of("a", "b", "c").stream();

stream.forEach(System.out::println);

// Error: el stream ya fue consumido
stream.count();
```

API Stream – orígenes de datos

- Un Stream siempre parte de un origen. Ese origen puede ser una colección, un array, un rango, un fichero, una secuencia generada o cualquier fuente capaz de producir elementos.

```java
// Ejemplo con colección
List<String> nombresL = List.of("Ana", "Luis", "Marta", "Pedro");
Stream<String> streamL = nombresL.stream();
```

```java
// Ejemplo con array
String[] nombresA = { "Ana", "Luis", "Marta" };
Stream<String> streamA = Arrays.stream(nombresA);
```

```java
// Ejemplo con rango numérico
IntStream numeros = IntStream.range(1, 10);
```

```java
// Ejemplo con generación
Stream<Double> aleatorios = Stream.generate(Math::random);
```

Obtener instancias de Stream

- Stream.of(T…): Stream<T>

  A partir de una lista de valores (varargs/array)

  Si la lista contiene literales numéricos, genera un Stream de wrappers, a diferencia de IntStream.of(int…), LongStream(long…) o DoubleStream.of(double…) que lo generan de primitivos

- *Arrays.stream(T[]):Stream<T>*

  A partir de un array existente

- *Collection<E>.stream():Stream<E>*

  A partir de una colección

- *Stream.iterate(T, UnaryOperator<T>):Stream<T>*

  Stream infinito, ordenado y secuencial a partir del valor inicial T y de aplicar la función pasada por parámetro *UnaryOperator. limit(xx)*

Operaciones intermedias – Filtrado (selección)

- *filter(Predicate<T>):Stream<T>*

  Retorna un Stream que contiene sólo los elementos que cumplen con el predicado pasado por parámetro

- *distinct():Stream<T>*

  Retorna un Stream sin elementos duplicados. Depende de la implementación de *equals(Object):boolean*

- *limit(long):Stream<T>*

  Retorna un Stream cuyo tamaño no es mayor al número pasado por parámetro. Los elementos son cortados hasta ese tamaño

- *skip(long):Stream<T>*

  Retorna un Stream que descarta los primeros N elementos. Si contiene menos elementos que N, entonces retorna un Stream vacío.

Operaciones intermedias – Ordenar los elementos

- *sorted():Stream<T>*

  Retorna un Stream que contiene los elementos ordenados en base a su orden natural. T debe implementar Comparable

- *sorted(*Comparator<? super T> comparator*):Stream<T>*

  Permite definir el orden personalizado con el Comparator

  Comparator.reverseOrder()

  Comparator.comparing(Persona::getNombre)

```
userList.stream().sorted(Comparator.comparing(User::getApellido)
                                   .thenComparing(User::getNombre))
```

Operaciones intermedias – Mapeo (transformación)

- *map(Function<T, R>): Stream<R>*

  Retorna un Stream que contiene el resultado de aplicar la función pasada por parámetro a todos los elementos del Stream. Transforma los elementos del tipo T al tipo R

- *mapToDouble(ToDoubleFunction<T>): DoubleStream*

- *mapToInt(ToIntFunction<T>): IntStream*

- *mapToLong(ToLongFunction<T>): LongStream*

Operaciones con streams – Mapeo (transformación)

- *flatMap(Function<T, Stream<R>>):Stream<R>*
  - Permite transformar cada elemento del Stream en otro Stream y al final concatenarlos todos en uno solo. Es un poco confuso, pero pensemos que cuando tengamos un Stream<Stream<R>>, este método nos permitirá transformarlo en Stream<R>

- *flatMapToDouble(Function<T, DoubleStream>): DoubleStream*

- *flatMapToInt(Function<T, IntStream>): IntStream*

- *flatMapToLong(Function<T, LongStream>): LongStream*

Operaciones terminales

- Operaciones terminales cuyo propósito es el consumo de los elementos del Stream, por ejemplo: *foreach(Consumer<T>):void*

- Operaciones terminales que permiten obtener datos del Stream como: conteo, mínimo, máximo, búsqueda, y en general reducir el Stream a un valor.

- Operaciones terminales que permiten recolectar los elementos de un Stream en estructuras mutables como por ejemplo listas.

Operaciones para reducir el Stream a un valor

- *count():long*
- *max(Comparator<T>):Optional<T>*
- *min(Comparator<T>):Optional<T>*

- *Definida en DoubleStream*
  - *average():OptionalDouble*
- *Definida en IntStream*
  - *sum():int*

Operaciones para reducir el Stream a un valor

- *reduce(BinaryOperator<T>):Optional<T>*
  - Realiza la reducción del Stream usando una función asociativa
  - Toma los valores del stream y los "condensa" a uno solo, usando una operación acumulativa. Es muy común para sumas, productos, concatenaciones, búsquedas de máximos o mínimos, etc.
- findFirst: Optional<T>
- findAny: Optional<T>
- anyMatch(Predicate<T>):boolean
- allMatch (Predicate<T>):boolean
- noneMatch(Predicate<T>):boolean

Operaciones para recolectar

collect(Collector):Stream<T>

- Está diseñado para **transformar los elementos del stream en una estructura mutable de datos diferente**, como una lista, un mapa, un conjunto o incluso un solo valor resumido.
- *Es una operación de **reducción mutable***

- *La clase Collectors tiene muchos métodos estáticos que retornan un Collector e implementa la funcionalidad.* Contiene implementaciones de reducción útiles para acumular elementos, agruparlos, resumirlos o transformarlos al final de un Stream

Operaciones para recolectar (Collectors)

Obtener colecciones básicas desde un stream()

.collect(Collectors.toList())

.collect(Collectors.toSet())

.collect(Collectors.toCollection(Supplier))

.collect(Collectors.toUnmodifiableList())

.collect(Collectors.toUnmodifiableSet())

.collect(Collectors.toMap(Function k, Function v))

.collect(Collectors.toUnmodifiableMap(Function k, Function v))

Operaciones para recolectar (Collectors)

Agrupación

.collect(Collectors.groupingBy(Function))

```
// Agrupa empleados por su departamento
Map<Departamento, List<Empleado>> porDepto = emps.stream()
    .collect(Collectors.groupingBy(Empleado::getDepartamento));
```

.collect(Collectors.groupingBy(Function, Collector))

```
//Obtener el salario medio por departamento
Map<Departamento, Double> salariosPorDepto = emps.stream().collect(
    Collectors.groupingBy(
        Empleado::getDepartamento, Collectors.averagingDouble(Empleado::getSalarioBase)
    )
);
```

Operaciones para recolectar (Collectors)

Partición

.collect(Collectors.partitioningBy(Predicate))

```java
// Divide estudiantes en aprobados y reprobados
Map<Boolean, List<Estudiante>> aprobados = estudiantes.stream()
    .collect(Collectors.partitioningBy(e -> e.getNota() >= 5));
```

Operaciones para recolectar (Collectors)

Cálculos y Resumenes

.collect(Collectors.joining())

.collect(Collectors.joining(CharSecuence sep))

.collect(Collectors.joining(CharSecuence sep, CharSecuence pref, CharSecuence suf))

```
//Concatena los nombres de los empleados separados por ", " en un String
String resu =
    emps.stream()
    .map(e -> e.getNombre()).collect(Collectors.joining(", "));
```

Operaciones para recolectar (Collectors)

Cálculos

Collectors.counting()

Collectors.minBy(Comparator)

Collectors.maxBy(Comparator)

Collectors.summingDouble(ToDoubleFunction)

Collectors.summingInt(ToIntFunction)

Collectors.summingInt(ToIntFunction)

Collectors.averagingDouble(ToDoubleFunction)

Collectors.averagingInt(ToIntFunction)

Collectors.averagingLong(ToLongFunction)

Resumenes

Collectors.summarizingInt(ToIntFunction)

Collectors.summarizingLong(ToLongFunction)

Collectors.summarizingDouble(ToDoubleFunction)

Retornan respectivamente

- IntSummaryStatistics
- LongSummaryStatistics
- DoubleSummaryStatistics

Que tienen los métodos

- getCount()
- getSum()
- getMin()
- getMax()
- getAverage()

Operaciones para recolectar (Collectors)

Cálculos y Resumenes

Existen también las funciones reductoras de Stream, como count() o DoubleStream.average(), pero las de Collectors pueden actuar como un “sub-collector”, por ejemplo, asociándolas con Collectors.groupingBy().

Si sólo necesitamos terminar el stream con un conteo o una media, usar las funciones de stream.

```
//Obtener el salario medio por departamento
Map<Departamento, Double> salariosPorDepto = emps.stream().collect(
    Collectors.groupingBy(
        Empleado::getDepartamento, Collectors.averagingDouble(Empleado::getSalarioBase)
    )
);
```

Streams paralelos

- El objetivo de los parallel streams es mejorar el rendimiento al procesar muchos datos dividiendo la tarea en varios threads.

- *En Stream*
  - *parallel():Stream*
  - *sequential():Stream*
- *En Collection*
  - parallelStream():Stream

Streams paralelos

- El uso de Streams en paralelo requiere más trabajo internamente, por lo que no siempre terminará más rápido que Streams secuenciales.
- Uso de streams paralelos:
  - Cuando el número de elementos es muy grande y las operaciones son costosas.
  - Operaciones a evitar:
    - *limit(long):Stream<T>*
    - *findFirst():Optional<T>*
    - *sorted():Stream<T>*
    - *distinct():Stream<T>*
  - Mejores estructuras a descomponer
    - ArrayList y arrays (ideal)
    - HashSet